WATTSINN · ENGINEERING · OCTOBER 2026

Your role at Wattsinn and a short work sample

For Ashiq Ali Abdul Lathief, following our conversation. Questions are welcome at any time.

What we build

Wattsinn runs an energy management platform for commercial and industrial sites in Germany. On site, a small industrial PC (the *edge device*) reads inverters, battery systems, meters, heat pumps and heating rods over Modbus TCP/RTU, SunSpec, HTTP and OCPP. It sends measurements to our cloud and executes schedules: when to charge or discharge a battery, when to heat, how to keep the grid connection below its peak limit.

The cloud side optimises these schedules with linear programming, using day-ahead prices, PV and load forecasts. The edge side has to execute them safely, also when the internet connection is gone.

Your role and first topics

You would join as an edge and integration engineer. The work sits between control engineering, communication protocols and real hardware, which matches your thesis and lab work well.

Topic	What it means	When
Telecontrol (Fernwirktechnik)	An IEC 60870-5-104 interface on the edge device, so that direct marketers and grid operators can read plant values and send power limits. First in simulation, then a pilot site.	Months 1–4
Device connectors	New devices via Modbus or HTTP: register maps, decoding, automated tests, verification on the real device.	Ongoing
Schedules (Fahrpläne)	Improve how optimised battery and heating schedules are built, delivered and executed on the edge device, including fallback behaviour.	From month 2
New hardware	Pilot of the Revolution Pi as an additional edge platform (RS485, digital I/O for power stages).	As needed
Plant controller (EZA-Regler)	A study in simulation first. A plant controller under VDE-AR-N 4110 is a certified, safety-relevant product. Nothing of it runs on a real plant before review, certification questions and a pilot are settled.	Later

You would also join commissioning on site, together with a qualified electrician, to learn the installations and devices we work with.

Tech stack

- **Edge:** Python 3.12, asyncio, pymodbus, MQTT, SQLite buffer, Docker on ARM64 Linux
- **Cloud:** FastAPI, PostgreSQL with TimescaleDB, Redis, Celery, HiGHS LP solver, AWS
- **Frontend:** React and TypeScript
- **Process:** GitHub, pull requests, CI with ruff and pytest on every change

How we work

- **A wrong value is worse than no value.** If data is stale or uncertain, we show it as such. We never fill gaps with guesses.
- **Plants first.** Anything that writes to a device needs limits, a defined behaviour on communication loss, and a review before it reaches a real site.
- **Tests prove the fix.** Every guard gets a test, and we check that the test fails when the fix is removed.
- **Simulation before hardware.** The same approach you described from your thesis.

WORK SAMPLE

Mini telecontrol gateway

A small, self-contained task close to your first topic. It is fictional and will not be used in production. Please spend **no more than 5–6 hours** on it. Incomplete is fine, as long as you write down what is missing.

Scenario

A PV inverter is reachable via Modbus TCP. A direct marketer wants to read its values and send active power limits via IEC 60870-5-104. Your gateway sits in between.

Inverter registers (simulate them, e.g. with a pymodbus server)

Address	Type	Content	Scale / unit	Access
40000	int32, big endian	Active power	1 W	read

40002	int32, big endian	Reactive power	1 var	read
40004	uint16	Grid frequency	0.01 Hz	read
40005	uint16	Status: 0 off, 1 running, 2 curtailed, 3 fault	enum	read
40010	uint16	Active power limit	1 %, valid 0–100	read / write

All holding registers, unit ID 1, rated power 100 kW.

IEC 104 mapping

IOA	Type	Content
1001	M_ME_NC_1	Active power in kW
1002	M_ME_NC_1	Reactive power in kvar
1003	M_ME_NC_1	Frequency in Hz
1004	M_ME_NC_1	Active power limit currently set, in %
1005	M_ME_NB_1	Status (enum as above)
5001	C_SE_NC_1	Command: active power limit in %

Requirements

1. Poll the inverter every second and convert the values to the IEC 104 units above.
2. If a value is older than 5 seconds, mark it as invalid. Do not keep sending the old value as if it were fresh.
3. Accept a limit command only in the range 0–100 %. Write it, read it back, and confirm the command only if the read-back matches. Otherwise send a negative confirmation.
4. Decide what happens if the IEC 104 connection is lost for more than 60 seconds, or if writing to the inverter fails. Make it configurable and justify your default in the README.
5. Write automated tests for scaling, validation, stale data and the fallback behaviour.

Part A (required): Modbus side, conversion, validation, fallback logic and tests. You may trigger the command from a test or a small CLI instead of real IEC 104.

Part B (optional): A real IEC 104 server, for example with the open-source Python library `c104`, plus a short demo against a 104 client.

Please send

- A Git repository or a zip file with the code and tests
- A README with: how to run it, your design decisions, and what would be missing for use on a real plant
- Short answers (half a page in total) to two questions:
  1. What changes if the gateway does not only pass through limits, but itself controls the active power at the grid connection point (a plant controller)? What makes that harder?
  2. How would you test the gateway on a real plant for the first time without risking the plant or the customer's revenue?

What we look at

Clean and safe behaviour in error cases, tests that really prove something, clear reasoning in the README, and an honest list of what is missing. Feature completeness matters less. English is fine for code, README and answers.

Please send your result within one week. We will then go through it together in a 45-minute call, where you show the code and explain your decisions.